Learning to Recover: Evaluating Recovery-Aware Fine-Tuning for Gemma 4

Comparing Success-Only and Recovery-Aware QLoRA Training for Python Code Repair

Vishnu
University of Massachusetts Amherst

November 12, 2026

Abstract

We ask whether a small coding language model learns to *recover* from failed code more effectively when fine-tuned on failure-execution-feedback–correction examples than when trained on successful code alone. Using Gemma 4 E4B-it, we collect visible-test trajectories, which record the model's code attempts, test results, and subsequent corrections using available test cases and independently train two 4-bit QLoRA adapters: success-only (C1) and recovery-aware (C2), each with 55 examples and exactly 5,963 supervised completion tokens. The original Gemma 4 model (C0), without fine-tuning, is also evaluated to measure whether fine-tuning improves performance. During development testing, we gave both models the same incorrect code and test feedback. The recovery-trained model (C2) fixed 5 out of 29 problems (17.2%), while the success-only model (C1) fixed 4 out of 29 (13.8%). This represents a small improvement of one additional problem. **However, this improvement was not seen in the final evaluation of 70 MBPP+ problems. Among the 24 incorrect solutions, both C1 and C2 fixed 2 (8.3%), while the original model (C0) fixed 3 (12.5%).** In our development experiments with 30 problems and up to three attempts, all three models produced correct final solutions for 16 out of 30 problems. Detailed error feedback helped C2 fix more problems than a simple retry instruction (5/24 vs. 3/24), but the difference was inconclusive. We also found inconsistencies between visible and official tests. Overall, our results suggest that more diverse recovery examples and reliable test feedback are needed to improve performance.

1 Introduction

Agentic coding systems need to do more than generate a first answer: they must interpret failed tests, correct their code, and stop making changes before they introduce new errors. Using test feedback to help small AI models fix coding mistakes is important when computing resources are limited. Offline supervised fine-tuning on successful solutions is straightforward, but its targets contain no explicit failed attempt or execution feedback. Recovery-oriented training instead conditions a corrected solution on the model's preceding error and the observable test result.

We study whether that additional supervision produces a measurable benefit *under a small, matched training budget*. This is motivated by prior work finding that offline correction traces need not translate into self-correction behavior [3], while execution-feedback RL can improve iterative synthesis [4]. We restrict scope to only Python function synthesis and revision, rather than repository-level bug localization, file editing, or SWE-bench.

Our analysis separates failures in the available tests, correctness according to official MBPP+ tests, and correctness of the final code returned by the model. We present results from both development and final evaluation, including cases where no improvement was observed.

2 Research Questions

Table 1: Research questions investigated in this study.

RQ	Focus	Research Question
RQ1	Recovery performance	Does recovery-aware QLoRA outperform success-only QLoRA on identical failed starting states?
RQ2	Failure types	Which types of coding errors does recovery-focused training help fix the most?
RQ3	Efficiency	Does the recovery condition avoid repeated failed attempts or reduce tokens and test executions?
RQ4	Generalization	Does recovery-focused training improve performance when the model has limited attempts and is tested on new programming problems?

Table 2: Training configurations used in our study.

Model	Training Type	Training Data
C0	Base model	Original Gemma 4 E4B-it model without any additional training [9].
C1	Success-only QLoRA	50 successful model solutions and 5 reference solutions.
C2	Recovery-aware QLoRA	27 successful model solutions, 4 reference solutions, 7 model self-recoveries, and 17 reference-based corrections.

3 Methods

3.1 Problem Pool and Trajectory Collection

MBPP provides natural-language function specifications, reference implementations, and original visible tests [1]; the EvalPlus framework supplies more extensive scoring for MBPP+ and HumanEval+ [2]. We exclude 108 overlapping MBPP+ IDs from an initial 374-task MBPP training partition, retaining 266 eligible training problems. We selected 40 programming problems and ran the original Gemma model twice on each problem, allowing up to three attempts per run using the available test cases. Across 80 trajectories, 53 first attempts pass visible tests; nine of 27 initially unsuccessful trajectories subsequently pass. The initially failing attempts comprise 22 wrong-output, four syntax, and one runtime failure. The dataset also includes 17 verified-reference corrections. All solutions were checked using the original test cases, but passing these tests does not guarantee that they would pass the more extensive MBPP+ tests.

3.2 Training Conditions

For recovery examples, the input includes the programming problem, previous incorrect code, and test feedback. The expected output is corrected Python code. The input prompt is excluded from the training loss.

Both C1 and C2 are independently fine-tuned from C0. C2 is not trained on top of C1.

We fine-tune both models using 4-bit QLoRA with NF4 quantization, FP16, LoRA rank 8, scaling factor 16, and a learning rate of 10^{-4}. Training uses a maximum sequence length of 1,024 tokens, batch size 1, and gradient accumulation of 4. Each model is trained for one epoch (14 optimizer updates) using seed 42.

Both models use 55 training examples and 5,963 supervised target tokens, although their input lengths and training data differ. Training takes 54.4 seconds for C1 and 73.9 seconds for C2 on a single NVIDIA T4 GPU [7, 8].

Figure 4 shows the training loss across optimizer steps. Since C1 and C2 use different training examples, their loss values cannot be directly compared.

3.3 Evaluation and Metrics

We evaluate the models on 180 distinct MBPP+ programming problems, divided into 110 development problems and 70 final test problems. The final test problems are kept separate to evaluate performance on unseen tasks.

For each problem, the original model (C0) generates an initial solution. If the code fails the available tests, C0, C1, and C2 receive the same incorrect code and error feedback, and each gets one attempt to fix it.

We use the more extensive MBPP+ tests to evaluate whether the corrected code is actually correct. These tests are not shown to the models during code generation. We also exclude initially correct solutions when calculating recovery rates.

We compare the models using recovery success rates and confidence intervals. In a separate experiment with 30 development problems, we allow up to three attempts and measure both the number of problems solved at any attempt and the correctness of the final returned code. Each attempt is limited to 384 generated tokens.

4 Results

4.1 RQ1: Recovery Performance

During development testing, 31 out of 110 initial solutions failed the visible tests, but two passed the official MBPP+ tests. Among the remaining 29 incorrect solutions, C2 fixed 5 (17.2%), while C0 and C1 each fixed 4 (13.8%). This shows a small improvement for C2, but the difference was not statistically significant.

In the final evaluation of 70 unseen problems, 35 initial solutions passed the official MBPP+ tests. Among the 24 incorrect solutions selected for recovery evaluation, C1 and C2 each fixed 2 (8.3%), while C0 fixed 3 (12.5%). Therefore, **the improvement observed during development did not continue in the final evaluation**, and recovery-focused training showed no clear advantage over success-only training.

4.2 RQ2: Failure Types

During development testing, most failures were incorrect outputs (29 out of 31), while only two were syntax errors and none were runtime errors. C2 fixed 7 out of 29 wrong-output errors, compared with 6 for C0 and C1. None of the models fixed the syntax errors. Since there

Table 3: Percentage of incorrect solutions fixed by each model in one attempt. All models receive the same starting code and test feedback. Solutions already correct under MBPP+ are excluded from the recovery-rate calculation.

Split	C0	C1	C2
Development (29)	4/29	4/29	5/29
Frozen holdout (24)	3/24	2/24	2/24

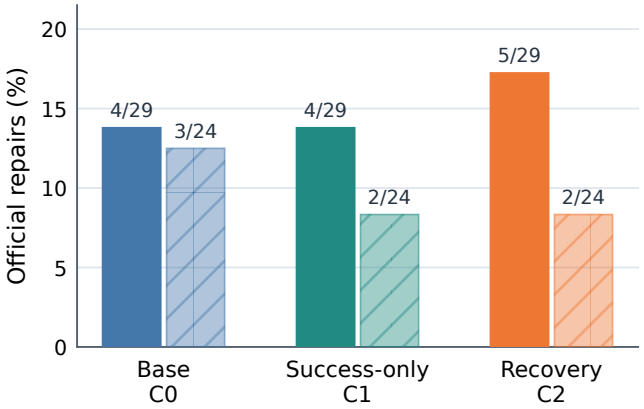


Figure 1: Comparison of recovery rates for C0 (blue), C1 (teal), and C2 (orange) on development and final test problems. Solid bars represent development results, while lighter bars represent final results. C2's small improvement during development was not observed in the final evaluation.

were few examples of other error types, we cannot confidently determine which types benefit most from recovery training.

4.3 RQ3-RQ4: Efficiency, Retries and Transfer

In our development experiment with 30 problems and up to three attempts, all three models produced correct final solutions for 16 out of 30 problems. However, some models generated correct solutions during earlier attempts but later changed them into incorrect ones (Fig. 3).

C0, C1, and C2 generated 380.4, 330.0, and 384.2 tokens per correctly solved problem, respectively. C1 used the fewest tokens and required slightly fewer test executions (Fig. 2). The models also occasionally repeated unsuccessful corrections. **Overall, recovery-focused training (C2) did not improve efficiency compared with success-only training (C1).**

We also tested the models on 20 HumanEval+ problems. C0 and C1 each solved 11 problems (55%), while C2 solved 12 (60%). However, this experiment evaluated only the first attempt without allowing corrections. Since the test set was small, we cannot conclude that C2 performs better on unseen programming problems.

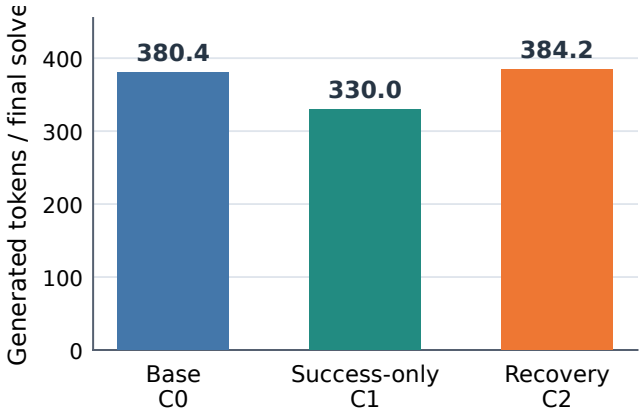


Figure 2: Comparison of tokens used per correctly solved problem across 30 development tasks, with up to three attempts each. All three models solved 16 problems. C1 used the fewest tokens, while C2 showed no improvement in efficiency.

Table 4: Comparison of the original MBPP test results and official MBPP+ results for C0's first code attempts.

Split	Local result	Official pass	Official fail
Development	Pass	62	17
Development	Fail	2	29
Final holdout	Pass	32	11
Final holdout	Fail	3	24

4.4 Feedback Specificity and Reliability

On 24 development problems, C2 fixed 5 errors using detailed test feedback, compared with 3 using a simple retry instruction. This suggests that detailed feedback may help, but the improvement was not statistically significant.

We also found differences between the original tests and the official MBPP+ tests (Table 4). During development, 19 solutions received different results from the two test sets, while 14 differed in the final evaluation. These differences can cause the model to make unnecessary changes to correct code. Our findings highlight the importance of reliable test feedback when training coding agents.

5 Discussion and Limitations

Our results show no clear improvement from recovery-focused training with a small dataset. However, this does not mean that recovery training is ineffective. C2 was trained on only seven genuine self-recovery examples, and both models received limited training. The small number of evaluation problems (29 development and 24 final) also makes it difficult to draw strong conclusions.

Another limitation is that the original MBPP tests sometimes give misleading feedback, which can cause unneces-

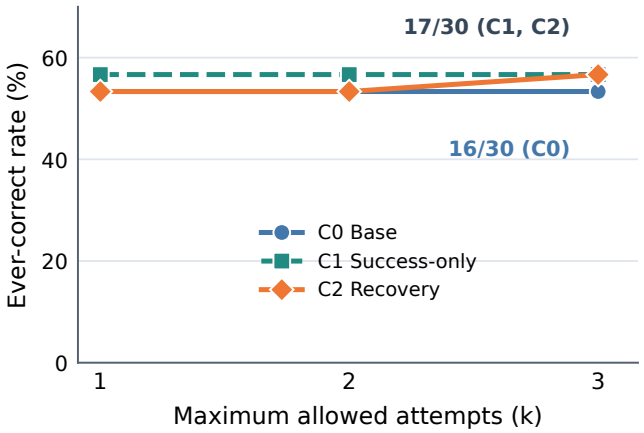


Figure 3: Comparison of the percentage of problems solved within one, two, or three attempts across 30 development test problems. C2 solved one additional problem on its third attempt, but all three models produced correct final solutions for only 16 out of 30 test problems (53.3%). The models did not have access to official MBPP+ test results while correcting their code.

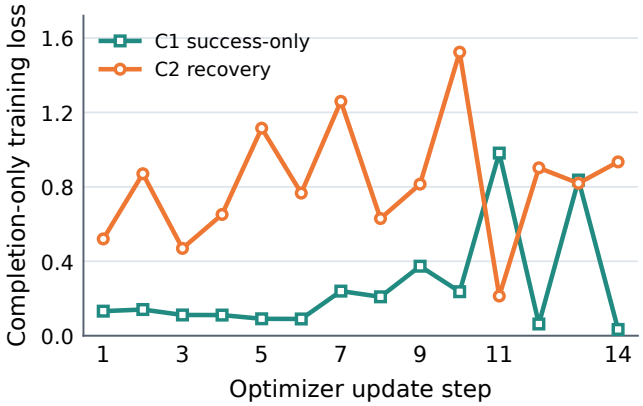


Figure 4: Training loss for C1 and C2 across 14 optimizer steps. The loss values vary between steps and cannot be directly compared because the models use different training data. This graph shows training loss, not validation performance.

sary corrections. We also tested the models on mistakes made by the original Gemma model, rather than on mistakes generated by each model itself.

Future work could improve performance by collecting more diverse and reliable recovery examples, using stronger tests to verify corrections, and experimenting with more training epochs and different training configurations. These improvements should be evaluated on new, unseen problems to measure their effectiveness.

6 Conclusion

Our comparison of success-only and recovery-focused QLoRA training for Gemma 4 E4B-it **showed a small improvement during development but no improvement in the final evaluation.** Both fine-tuned models fixed only 2 out of 24 incorrect solutions, and when given up to three attempts, all three models correctly solved 16 out of 30 development problems (53.3%).

Our findings suggest that recovery training requires more diverse and reliable examples. They also highlight the importance of accurate test feedback and evaluating the correctness of the final code returned by the model.

References

[1] J. Austin et al. *Program Synthesis with Large Language Models*. arXiv:2108.07732, 2021.

[2] J. Liu, C. S. Xia, Y. Wang, and L. Zhang. *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation*. NeurIPS, 2023.

[3] A. Kumar et al. *Training Language Models to Self-Correct via Reinforcement Learning*. ICLR, 2025.

[4] J. Gehring et al. *RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning*. ICML, 2025.

[5] N. Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS, 2023.

[6] M. Kim. *How to Learn from Negative Trajectories in Coding Agents*. Microsoft Debug-gym Research Blog, August 2026. `https://microsoft.github.io/debug-gym/blog/2026/08/negative-pi/`.

[7] E. J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.

[8] T. Dettmers et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS, 2023.

[9] Google DeepMind. *Gemma 4 Model Card*, 2026. `https://ai.google.dev/gemma/docs/core/model_card_4`.